CloudGuard

Hybrid Cloud Configuration Change Impact Analyzer

Complete Viva Script

Part 1 Student Side — Blast-Radius Guard

Part 2 Professional Side — CloudGuard PR Bot

Shobaa · September 2026

Viva Script — Student Side (Blast-Radius Guard)

Project: CloudGuard — Hybrid Cloud Configuration Change Impact Analyzer **This part:** the student side. It has a Chrome/Edge extension for the AWS Console, a guard for the AWS CLI and boto3, and the CloudGuard website. **Repo:** github.com/Shobaa-cloud/HybridStudents

> How to use this script - Lines in quote blocks marked **Say:** are meant to be spoken out loud, close to word for word. - Everything else is background you should understand so you can answer follow-up questions in your own words. - Section 14 is the question bank. Practise answering those out loud without reading. - Section 16 maps each file to what it does. Examiners often say "show me where that happens in the code". Know those files.

---

1. Opening (about 45 seconds)

> **Say:** "Our project is CloudGuard, a hybrid cloud configuration change impact analyzer. Its job is to tell you what a change will break **before** you make it, not after.
>
> We built it for two audiences, because they work in completely different ways. Professionals change infrastructure through Terraform code and pull requests, so for them we built a PR bot. Students and freshers don't write Terraform. They click buttons in the AWS Console or run a few CLI commands, so for them we built the part I'm presenting now: the **Blast-Radius Guard**.
>
> It's a browser extension, plus a small Python hook for the CLI and SDK. It pauses a risky action, like terminating an EC2 instance or opening SSH to the whole internet, and explains in plain English what will happen and what else it affects. Then the student decides: cancel, or proceed. It needs no AWS keys, no server and no setup beyond loading the extension."

---

2. How we got the idea (our story)

> **Say:** "When we were freshers, we started building things in AWS without full technical knowledge. We were honestly scared to touch or change any configuration out of curiosity, because we thought we'd break something or get billed. A friend of ours actually got an AWS bill after she thought she had deleted everything, because something was still running that she hadn't noticed.
>
> We also didn't want to ask an AI every time before clicking something. That fear stopped us from experimenting, and experimenting is how you actually learn the cloud.
>
> Tools like AWS Config and CloudTrail do exist, but they tell you what changed **after** it has happened. Terraform plan does show a preview before applying, but it's a raw technical diff that assumes you already understand what it means, and beginners don't use Terraform anyway. So we wanted something that says **what will happen, in plain English, before the mistake happens**, right where the student is clicking. And we wanted the same idea to be useful in industry too, which is why there's a professional side."

Why this story is strong: it's a real problem (fear of breaking things, fear of billing), and it leads straight to the design decisions: warn before the action, plain English, zero cost, zero credentials.

Be careful here: don't say "terraform plan only shows things after". It shows a preview before apply. The correct point is that it's **technical, not plain English, and students don't use Terraform**.

---

3. Problem statement (one paragraph)

Beginners learning AWS through the Console make a small set of very common, very costly mistakes: terminating an instance and losing its disk, leaving an Elastic IP that keeps billing, and opening SSH, RDP or database ports to `0.0.0.0/0`, the entire internet. AWS's own protections are either generic ("Are you sure?"), or small warning text that doesn't stop the save, or services like AWS Config that detect a problem after it exists and cost money to run. There is no free, zero-setup tool that **intercepts the action before it runs** and explains its **blast radius** (what else it affects) in language a beginner understands.

---

4. What we built (three pieces)

Piece	Who uses it	What it does
CloudGuard website (`website/`)	Everyone	Landing page that asks "Who's here?". **I'm a student** leads to the extension download and a 4-step install guide. **I'm an intern or pro** leads to the PR bot. This is the "one application, two roles" idea.
Browser extension (`aws-blast-radius-extension/`)	Students using the AWS Console	Intercepts two risky clicks, shows a warning with the blast radius, and lets the student cancel or proceed
CLI/SDK guard (`aws-blast-radius-cli-guard/`)	Students using `aws` CLI v1 or `boto3` Python scripts	Same idea for the terminal: pauses the API call and asks `Proceed anyway? [y/N]`

The two mistakes we catch (deliberately small MVP scope): 1. **Terminating EC2 instances.** Termination is permanent, the root EBS volume is deleted by default, instance-store data is lost, and an Elastic IP keeps billing. 2. **Saving inbound security-group rules open to `0.0.0.0/0` or `::/0`.** SSH, RDP, databases and "All traffic" are **High risk**. Other ports get a **Heads up**. Web ports 80 and 443 pass silently, because websites are meant to be public.

---

5. The mechanism, named

We gave both sides a formal name so they're parallel:

Side	Mechanism
Professional	**Pre-deployment static blast-radius analysis of Infrastructure-as-Code**
Student	**Pre-execution dynamic blast-radius analysis of live API calls**

> **Say:** "The professional side is **static**: it reads a Terraform plan file before deployment, without running anything. The student side is **dynamic**: there's no file, so the 'change' is the actual action the student is performing, captured a split second before it runs. Same concept, blast radius before the change, but a different trigger."

Why it's called "blast radius": like an explosion, a change affects more than the thing you touched. Opening a security group affects every instance that uses that group.

---

6. How the browser extension works, step by step

1. **Injection.** `manifest.json` (Manifest V3) tells Chrome to inject `inventory.js`, `content.js` and `modal.css` into every page matching `https://*.console.aws.amazon.com/*`, including frames (`all_frames: true`).
2. **Listening.** `content.js` adds a click listener on the document in the **capture phase** (`addEventListener("click", onClick, true)`). Browser events travel down from the page to the button (capture) before bubbling back up. AWS's React code listens in the bubble phase, so **our listener runs first**.
3. **Identifying the button.** It uses `e.composedPath()` to find the real button, even inside Shadow DOM, and reads its visible **text** ("Terminate", "Save rules").
4. **Checking the risk.** - `checkTerminate` : only if the button is inside AWS's confirmation dialog. It reads instance IDs (`i-xxxxxxxx`) **only from that dialog**, so the warning always names the instance actually being terminated. - `checkInboundRules` : only on the **Edit inbound rules** page. It finds every rule row whose source is `0.0.0.0/0` or `::/0`, reads the port, and grades it with `assessRule`.
5. **Blocking.** If something is risky, it calls `e.preventDefault()` and `e.stopImmediatePropagation()`, so **AWS never receives the click**.
6. **Explaining.** It shows our modal: risk badge, resource IDs, reasons, the blast-radius box, a tip (for example "Use Stop instead" or "Change Source to My IP"), and two buttons. **"Cancel, keep me safe" is focused by default**, so Enter or Escape is the safe choice.
7. **Deciding.** - **Cancel:** nothing happens, and the "mistakes avoided" counter goes up. - **Proceed anyway:** we set `bypassNextClick = true` and call `button.click()` again. That re-click passes straight through to AWS, so **AWS does the action itself, under the student's own login session**. The extension never makes an AWS call.
8. **Stats.** `chrome.storage.local` keeps "warnings shown" and "mistakes avoided", which the toolbar popup displays.

How the blast radius is found without AWS keys (`inventory.js`)

This was the answer to "AWS already shows a warning, so what do you add?". We wanted to show **what else is affected**, not just "this is risky".

- While the student browses normally, `inventory.js` reads the tables **the console is already showing**:

- **EC2 → Instances:** each instance's ID, name, state, public IP and security groups
- **Security Groups list:** which group ID has which name
- It stores this in `chrome.storage.local`, in the student's own browser only. It sends nothing anywhere and makes no API calls.
- When the student opens SSH on `sg-abc`, the warning says, for example, **"Blast radius: 2 instances use this security group"**, lists them, and highlights which ones have a public IP and **can be reached from the internet as soon as you save**.
- It's honest about freshness: "Based on the Instances page as you last saw it (5 min ago)." If it hasn't learned anything yet, it says "Open EC2 → Instances once, and next time the guard will list exactly which instances this change reaches."
- The popup has a **Forget** button that wipes the stored inventory (privacy control).

Tricky engineering problems we solved (good viva material)

Problem	What we did
The Security Groups page lives inside a same-origin **iframe** (`security-groups-react-frame`) that Chrome didn't inject into, so no warning appeared	`forEachDoc` walks into same-origin iframes from the top page and attaches the guard there, re-checking every second because frames get rebuilt as you navigate
Risk of **two modals** when a frame is guarded twice	A `data-brg-guarded` marker on the frame's DOM so it's only guarded once
AWS changes CSS class names often	We detect by **visible button text** and **resource-ID patterns** (`i-…`, `sg-…`), which survive redesigns much better
The source CIDR sometimes appears as an input value, sometimes as a "chip" label	We check both inputs and text nodes, then climb up to the rule row (`findRuleRow`)
Page text is untrusted (XSS risk)	The modal uses `textContent`, never `innerHTML`
Our CSS clashing with AWS's	Every class is prefixed `brg-`

---

7. How the CLI/SDK guard works, step by step

> **Say:** "At first we planned only a CLI wrapper. Then we asked: why only the CLI, and not the SDK? AWS CLI v1 and boto3 scripts both run on the same library, **botocore**. So instead of two tools, we hook botocore once and catch both."

1. **Auto-loading.** `sitecustomize.py` is a special file Python runs automatically at startup if it's on `PYTHONPATH`. It calls `enable()`. The student changes no code.
2. **Patching.** `hook.py` replaces `botocore.session.Session.create_client` with our version (monkey-patching). Every EC2 client created by the CLI or boto3 goes through it.
3. **Registering.** For watched operations (`TerminateInstances`, `AuthorizeSecurityGroupIngress`), we register a handler on botocore's own event system: `before-parameter-build.ec2.TerminateInstances`. botocore fires this event **before the request is built and sent**.
4. **Checking.** `rules.py` runs the check: - Terminate: calls `describe_instances` and `describe_addresses` (read-only) to list the **exact EBS volumes with DeleteOnTermination=true** and any Elastic IP. - Ingress: flags `0.0.0.0/0` and `::/0` on sensitive ports.
5. **Confirming.** It prints a warning banner and asks `Proceed anyway? [y/N]`. **No is the default.**
6. **Aborting.** If the answer isn't `y`, it raises `BlastRadiusAborted`. An exception inside that botocore event **cancels the call**, so AWS never receives it.

Why the CLI guard can be more precise than the extension: it runs with the student's own CLI credentials (which they already have configured), so it can make read-only describe calls and name the exact volume IDs. The extension deliberately has no credentials, so for termination it describes AWS's default behaviour.

---

8. Tools and techniques, with why and why not

Tool / technique	Why we used it	Alternative we rejected, and why
Chrome extension, Manifest V3	Runs right where students work (the Console). MV3 is the current standard, and MV2 is being phased out. Works in Chrome, Edge and Brave.	**Desktop app:** can't see what's happening inside the browser. **Separate website:** students would have to copy things over manually, too much friction.
Content script (DOM-level interception)	Can see the button, the dialog and the form, and can stop the click before AWS's code runs	**Blocking network requests (`webRequest`):** MV3 removed blocking `webRequest` for normal extensions, and `declarativeNetRequest` can't read request bodies, so we couldn't tell what's in the request. Also, by the time a request is sent it's too late to explain nicely.

Tool / technique	Why we used it	Alternative we rejected, and why
Capture-phase event listener	Runs **before** AWS's own React handlers, so the action is truly stopped **before** it happens	Bubble phase: AWS would get the click first
`preventDefault` + `stopImmediatePropagation`, **then re-click on Proceed**	We never perform the action ourselves. AWS does it under the student's session.	Calling AWS APIs from the extension would need credentials
`chrome.storage.local`	Local only, survives restarts, needs only the `storage` permission	A backend database would mean sending data off the student's machine
Text and ID-pattern detection (regex)	Survives AWS UI redesigns better than CSS classes	CSS selectors break whenever AWS renames a class
Plain JavaScript, no framework or build step	Tiny (about 24 KB), easy to audit, zero setup	React or TypeScript would add a build step for no user benefit
Python + botocore event hooks	One hook covers **CLI v1 and boto3**, and uses botocore's official event system	A **CLI wrapper** only catches CLI commands, not SDK scripts
`sitecustomize.py` + `PYTHONPATH`	Auto-loads the guard with **zero code changes** in the student's scripts	Asking students to import our module in every script
Rule-based checks	Every warning can be explained by a specific rule, and it's instant, free and offline	**Machine learning:** there's no labelled dataset of "this click caused this incident", and a black box isn't trustworthy for a safety tool
Scoped test IAM user (for testing the CLI guard)	Least privilege: only the describe, terminate and authorize actions needed for testing	Using root or admin keys
Static website (HTML/CSS/JS)	Role picker plus download and install guide, can be hosted anywhere for free	A heavy app framework wasn't needed
`package_extension.py`	One command rebuilds the downloadable zip so the website always serves the latest version	Zipping by hand each time (error-prone)

9. Risk logic (know these numbers)

Sensitive ports (open to the internet means **High risk**): 22 SSH · 3389 RDP · 3306 MySQL · 5432 PostgreSQL · 1433 SQL Server · 27017 MongoDB · 6379 Redis · 9200 Elasticsearch (The CLI guard's rule list is a slightly shorter subset.)

Rule	Result
"All traffic", "All TCP", "All UDP", or ports 0–65535 open to the world	**High:** every service is exposed
A sensitive port open to the world	**High:** "bots scan the whole internet for this port within minutes"
Port 80 or 443 open to the world	**No warning**, because websites are meant to be public
Any other port open to the world	**Heads up** (medium)
Outbound rules	Not checked. Outbound to `0.0.0.0/0` is AWS's normal default.

10. Live demo plan (what to click)

Before the viva: extension loaded in Chrome, logged in to your free AWS account, one `t2.micro` / `t3.micro` instance running with its own security group. Open **EC2** → **Instances** once and **Security Groups** once so the inventory is learned.

Demo 1: open SSH to the world 1. EC2 → Security Groups → select your group → **Edit inbound rules** 2. **Add rule** → Type **SSH** → Source **Anywhere-IPv4** 3. Click **Save rules**, and the **High risk** modal appears 4. Point out: the port and reason, **"Blast radius: 1 instance uses this security group"**, which instances are internet-reachable, and the "My IP" tip 5. Click **Cancel, keep me safe**, then refresh the page and show the **rule was not saved**. This proves it acts **before** the action.

Demo 2: terminate 1. Instances → select instance → Instance state → **Terminate** → click **Terminate** in AWS's dialog 2. Our modal appears: permanent, root disk deleted, Elastic IP keeps billing, tip "use Stop instead" 3. Cancel. Show the instance is still running.

Demo 3: popup. Click the toolbar icon to show "warnings shown / mistakes avoided" and the learned inventory.

Optional Demo 4: CLI (only show this after you've tested it yourself). `aws ec2 authorize-security-group-ingress ... --port 3306 --cidr 0.0.0.0/0` shows the banner, answer `n`, and you get `BlastRadiusAborted`.

After the demo, **delete the test resources** (zero billing).

---

11. Advantages (our points, polished)

1. **Warns before, not after.** The click is stopped in the capture phase, before AWS receives it. AWS Config and CloudTrail only report after the change exists.
2. **Explains the blast radius, not just "risky".** It names the instances that use the group and which ones become internet-reachable. AWS's own inbound-rule note is generic text that doesn't stop the save and doesn't say what's affected.
3. **Zero credentials.** It never asks for, stores or sees AWS keys. Its only permission is `storage`. So there's **no central server holding students' access that could get hacked**, unlike SaaS tools that need a role in your account.
4. **Zero cost, zero billing risk.** No AWS Config (which charges per configuration item), no Lambda, no servers. This mattered to us because of the billing story.
5. **Zero setup.** No AWS Config, no Terraform, no terminal, no IaC knowledge. A fresher doesn't need to learn those tools first. About a minute to install.
6. **Protects curiosity instead of blocking it.** It never forbids anything. "Proceed anyway" always works, so students can still experiment, just informed. It teaches why ("bots scan port 22 within minutes") and the safer alternative ("Stop instead of Terminate", "My IP").
7. **Privacy.** The inventory stays in the student's browser, and there's a Forget button.
8. **Covers Console, CLI and SDK.** One botocore hook covers both CLI v1 and boto3 scripts.
9. **Safe by default.** Cancel is focused, Escape cancels, clicking outside cancels. In the terminal, `N` is the default.
10. **Resilient.** Detection uses visible text and ID patterns, not fragile CSS classes.
11. **Measurable.** The popup counts warnings and mistakes avoided.

---

12. Limitations (say these confidently; honesty scores marks)

1. **AWS only on the student side.** Azure (`portal.azure.com`) and GCP (`console.cloud.google.com`) have completely different page layouts, so each needs its own checks. The core (catch click → check → warn → count) is reusable. The **multi-cloud part of the project lives on the professional side**, which handles AWS, Azure and GCP in one graph.
2. **Only two scenarios.** Terminate and inbound rules. This was a deliberate MVP focusing on the most costly fresher mistakes. More (public S3 buckets, deleting security groups, IAM changes) would follow the same pattern.
3. **Depends on the console's page layout.** If AWS redesigns the page, detection may need adjusting. We reduced this by matching text and IDs instead of CSS classes.
4. **The inventory is only as fresh as what the student last viewed.** The warning shows how old the data is. Matching groups by name is a fallback, and two VPCs can both have a group called "default".
5. **The terminate warning in the extension describes AWS default behaviour**, not this instance's exact volumes, because the extension has no credentials. (The CLI guard does name exact volumes.) A future "deep scan" could use short-lived STS credentials from a backend, never long-lived keys in the browser.
6. **AWS CLI v2 is not covered.** It's a compiled binary with its own bundled Python, so our botocore hook in the student's environment doesn't reach it. CLI v1 and boto3 are covered.
7. **The CLI prompt is interactive** (`input()`), so it's not suited to unattended scripts or CI without an allowlist or `--yes` option.
8. **Not on the Chrome Web Store yet**, so install is 4 steps (Developer mode → Load unpacked). Publishing costs a one-time $5 fee plus review, and would turn install into one "Add to Chrome" click.
9. **It can be bypassed.** A student can disable it or click Proceed. It's a **learning seatbelt, not a security control**, by design.

---

13. Future scope

- Azure and GCP console support (the same engine with new URL patterns and checks)
- More scenarios: public S3 bucket, deleting a security group still in use, `0.0.0.0/0` on IPv6, IAM policy with `*`
- Chrome Web Store publishing for one-click install
- Optional "deep scan" using short-lived STS credentials for exact volume and dependency data
- CLI v2 support through a wrapper around the v2 binary

---

14. Question bank (practise these out loud)

Q1. What is your project in one line? A tool that tells you what a cloud change will break before you make it, in plain English: a PR bot for professionals and a browser guard for students.

Q2. Why is it called "hybrid"? Two meanings. It's **multi-cloud**: the professional side analyses AWS, Azure and GCP resources in one dependency graph, including changes that cross from one cloud to another. And it's **hybrid in audience**: one project with two front doors, students and professionals, chosen on the website's role picker.

Q3. AWS already shows a warning on 0.0.0.0/0. Why do we need yours? AWS's note is small generic text. It doesn't stop the save, says the same thing for every port, doesn't give a fix, and doesn't tell you what's affected. Ours pauses the save, names the port and why it's dangerous, suggests "My IP", and shows the **blast radius**: which instances use that group and which become reachable from the internet.

Q4. Does it really stop the action before it happens, or after? Before. The listener runs in the capture phase, before AWS's React handlers, and calls `preventDefault` and `stopImmediatePropagation`. We proved it: click Cancel, refresh, and the rule is not there. (Early in testing it looked like it warned after, but that rule had already been saved before the extension was installed. It even had a rule ID `sgr-…`, which only saved rules get.)

Q5. Does the extension need AWS credentials? No. It never asks for, stores or sees keys, and makes no AWS API calls. On Proceed, it re-clicks AWS's own button, so AWS acts under the student's existing login.

Q6. Then how does it know which instances use a security group? `inventory.js` reads the Instances and Security Groups tables the console is already showing while the student browses, and stores them locally. That's why it shows "based on the Instances page as you last saw it (x min ago)".

Q7. Why is that more secure than tools like Overmind? Hosted tools need standing access to your cloud account, usually a role you give their servers. If that vendor were breached, attackers could reach every customer's account: that's third-party or supply-chain risk. We hold nobody's access, so there's nothing central to steal. (Correct wording: the danger is **access roles or keys**, not account IDs. AWS doesn't treat account IDs as secret.)

Q8. Is it continuously monitoring the account 24/7? No, and that's deliberate. It's **event-driven**: it reacts at the moment of action (a click, or an API call). Nothing polls the account, so there's no cost and no credentials.

Q9. Why a browser extension and not a website or app? The risky action happens inside the AWS Console tab. Only an extension can see and pause that click. A separate website would need the student to copy things over, which nobody does mid-task.

Q10. Why Manifest V3? It's Chrome's current extension platform, and MV2 is being phased out. It also forced a good design: MV3 doesn't allow blocking network requests, so we intercept at the UI level, which is also the point where we can explain things in plain English.

Q11. Why intercept clicks and not network requests? MV3's `declarativeNetRequest` can block by URL but can't read the request body, so it can't tell what the request does. And by the time a request is sent, the user is gone from the decision. At the click we have context: the dialog, the form, and the instance IDs.

Q12. What is the capture phase? DOM events travel from the document down to the target (capture), then back up (bubble). Registering with `true` as the third argument means our listener runs on the way down, before AWS's handlers.

Q13. What happens when the student clicks Proceed? We set `bypassNextClick = true` and call `button.click()`. Our listener sees the flag, lets that one click through, and AWS's own handler runs normally.

Q14. Why only two scenarios? They're the two most common, most expensive fresher mistakes: permanent data loss plus leftover billing, and exposing SSH or databases to internet scanners. MVP first, done properly. The design makes adding more scenarios straightforward.

Q15. Why do ports 80 and 443 pass silently? Web servers are supposed to be public. Warning on them would create **alert fatigue**, and students would learn to click Proceed without reading.

Q16. Why does the CLI guard hook botocore instead of wrapping the CLI? CLI v1 and boto3 both run on botocore. One hook on `create_client` plus botocore's `before-parameter-build` event covers both, with no code changes, through `sitecustomize.py`.

Q17. What's monkey-patching, and is it safe? Replacing a function at runtime with our own version that calls the original. We keep the original `create_client`, call it, and just register extra handlers on the client. We mark the patch so it's never applied twice. `sitecustomize` wraps everything in try/except so a failure can never break an unrelated Python program.

Q18. Why doesn't it cover AWS CLI v2? v2 is a compiled standalone binary with its own bundled Python, not a botocore process in the student's environment, so our hook doesn't load into it. That's a documented scoping decision. A wrapper around the v2 binary is future work.

Q19. Why rule-based and not AI or ML? There's no labelled dataset of "this click caused this incident". Rules are explainable (every warning traces to one statable rule), instant, free, offline and deterministic. For a safety tool, trust matters more than sounding clever.

Q20. Why not just use AWS Config or Trusted Advisor? AWS Config reports after a resource is non-compliant, and it's billed per configuration item recorded and per rule evaluation, which doesn't suit a zero-billing student account. Neither stops the click, and neither explains in beginner language. Also, a fresher would have to learn and set up those services first.

Q21. Why not just teach students Terraform so they can use `terraform plan`? Freshers start in the Console. Asking them to learn IaC first is the barrier we're trying to remove. And `terraform plan` output is a technical diff, not "this will delete your disk". Students grow into Terraform later, and that's what the professional side is for.

Q22. What about Azure and GCP students? Honest answer: the student guard is AWS-only today, because each portal has a different page structure. The engine is reusable: only the URL match and the detection functions change. Multi-cloud analysis is done on the professional side, from Terraform plans.

Q23. Can the extension damage my AWS account? No. It can't create, change or delete anything. It only pauses a click and shows a message. AWS acts only if you choose Proceed.

Q24. What data does it store, and where? Counters, plus a small inventory (instance ID, name, state, public IP, group names), all in `chrome.storage.local` in the student's own browser. Nothing is sent anywhere. The Forget button clears it.

Q25. What if AWS changes its console design? We match visible text ("Terminate", "Save rules") and resource ID patterns, not CSS classes, which survives most redesigns. If something still breaks, `TERMINATE_LABEL` and `findRuleRow` are the two places to adjust.

Q26. What was the hardest bug? The warning didn't appear on the Security Groups page. That page is inside a same-origin iframe (`security-groups-react-frame`) that Chrome wasn't injecting our script into. We fixed it by walking into same-origin frames from the top page and attaching the guard there, with a marker so no frame gets two modals.

Q27. How did you prevent XSS in your modal? All text is set with `textContent`, never `innerHTML`, because page text such as instance names is untrusted.

Q28. How did you test it? In my free-tier AWS account: a `t2.micro` instance, a test security group, then both scenarios. Cancel means the rule and the instance are unchanged after refresh. Proceed means AWS completes it normally. Afterwards we deleted everything, so there was no billing. The CLI/SDK guard is a working prototype, but it hasn't been tested live against AWS yet; that's our next testing step.

Q29. What accuracy does it have? It's not a statistical model, so "accuracy percentage" isn't the right measure. It's deterministic: for the two supported scenarios on the current console layout, it triggered correctly in every test we ran. The honest limits are scope (two scenarios, AWS only) and inventory freshness. Those are coverage limits, not random errors.

Q30. Who benefits? Students, freshers, bootcamp and college lab users, anyone learning AWS on a personal or free-tier account. Also educators running labs, who want fewer "I got billed" or "my instance got hacked" stories.

Q31. How do students install it? Website → **I'm a student** → download the zip → unzip → `chrome://extensions` → Developer mode on → **Load unpacked**. On the Chrome Web Store it would be one click.

Q32. How does the student side relate to the professional side? Same idea, blast radius before the change, but a different input. Professionals give a Terraform plan in a pull request (static). Students give nothing: the live action itself is the input (dynamic). The website routes each person by role.

Q33. Did you use AI tools while building it? Answer honestly: yes, as a coding assistant, like most developers do today. But the problem, the decisions and the testing were ours, and I can explain every part of the code. Then name decisions that came from you (see the note at the end of this file).

Q34. What would you do with more time? Azure and GCP console support, more scenarios (public S3, IAM wildcards), Chrome Web Store publishing, and optional deep scan through short-lived credentials.

Q35. Isn't this just a pop-up? The pop-up is the output. The work is in deciding **when** to show it (only real risks, not 80/443, not outbound), **what to say** (the specific port, the reason, the fix), and **what else it affects** (the learned inventory linking groups to instances and public IPs), while stopping the action before AWS gets it, with no credentials.

---

15. Phrases to use, and traps to avoid

Don't say	Say instead
"Terraform plan only shows after"	"Terraform plan is a technical diff, and students don't use Terraform"
"It's live testing"	"It intercepts the live action before it runs"
"It's 100% accurate"	"It's deterministic, and within its two scenarios it triggered correctly in all our tests"

Don't say	Say instead
"It's AI"	"It's rule-based, which is deliberate for explainability"
"It works on all clouds" (student side)	"The student guard is AWS-only; multi-cloud is on the professional side"
"Hackers can steal account IDs from other tools"	"Hosted tools hold access roles to customers' accounts; we hold none"
"It blocks students"	"It pauses and explains; the student always decides"

16. Code map (know where everything is)

File	What it does	Key names to remember
`aws-blast-radius-extension/manifest.json`	MV3 config: matches `*.console.aws.amazon.com`, `all_frames`, only `storage` permission	`content_scripts`, `permissions`
`src/content.js`	Click interception, risk checks, modal	`onClick`, `checkTerminate`, `checkInboundRules`, `assessRule`, `showWarningModal`, `bypassNextClick`, `guard`, `forEachDoc`, `SENSITIVE_PORTS`
`src/inventory.js`	Learns instances and group names from console tables, with no API calls	`learnFrom`, `readTables`, `instancesUsingGroup`, `knownInstance`
`src/modal.css`	Modal styles, all `brg-` prefixed	
`popup.html/js/css`	Toolbar popup: counters, learned inventory, Forget button	`warned`, `avoided`
`aws-blast-radius-cli-guard/sitecustomize.py`	Auto-loads the guard at Python startup	`enable()`
`blast_radius_guard/hook.py`	Patches `create_client`, registers botocore events, prompt	`WATCHED`, `before-parameter-build`, `BlastRadiusAborted`
`blast_radius_guard/rules.py`	Terminate and ingress checks using read-only describe calls	`check_terminate_instances`, `check_authorize_ingress`
`website/index.html, app.js, styles.css`	Role picker, install guide, FAQ, PR bot section	
`package_extension.py`	Rebuilds `website/downloads/blast-radius-guard.zip`	

Note for you (not for the examiner)

Many of the strongest design decisions in this project came from **your own questions**. Use them as proof you own it: - "Why only CLI, why not SDK?" led to the botocore hook covering both. - "AWS already warns, so why us?" led to the real blast-radius inventory. - "Is it really before, not after?" led to the Cancel-then-refresh test that proves it. - "What about Azure and GCP?" led to naming the scope honestly. - "No one's credentials, so no vendor breach risk" led to the zero-credential design as a headline advantage. - The billing-fear story led to zero cost and zero AWS Config.

Viva Script — Professional Side (CloudGuard PR Bot)

Project: CloudGuard — Hybrid Cloud Configuration Change Impact Analyzer **This part:** the professional side, a GitHub Actions PR bot that analyses a Terraform plan before it's applied, comments the blast radius, risk, dependency graph, recommendations and rollback plan on the pull request, and **blocks the merge** when risk is CRITICAL. **Repo:** github.com/Shobaa-cloud/HybridCloudImpactAnalyzer_Bot (branch `demo/blast-radius-check`)

> How to use this script - Lines in quote blocks marked **Say:** are meant to be spoken out loud. - Sections 6 and 7 are the core of the project. You must be able to explain the pipeline and the formulas without notes. - Section 15 is the question bank. Practise out loud. - Section 18 maps each file to what it does.

---

1. Opening (about 45 seconds)

> **Say:** "In companies, infrastructure is changed through code, usually Terraform, and every change goes through a pull request. Before anyone clicks Apply, Terraform can produce a plan: a list of what will be created, changed or deleted. But that plan only says **what** changes. It doesn't say **what else will break** because of it, or how risky it is.
>
> CloudGuard's professional side is a **PR bot**. It reads that Terraform plan inside the company's own GitHub Actions pipeline, builds a **dependency graph** of the resources, traces the **blast radius** of every change, scores the **risk**, explains it in **plain English**, draws the dependency graph right in the PR comment, gives **recommendations** and a **rollback plan**, and if the risk is CRITICAL, it **fails the check and blocks the merge**.
>
> It works across **AWS, Azure and GCP** in the same plan, and it needs **no cloud credentials**, because it analyses the plan file, not the live account."

---

2. The problem

1. **Configuration changes cause outages.** Some of the biggest public outages were caused by configuration changes, not code bugs. For example, the 2017 Amazon S3 outage in us-east-1 started from a mistyped command that removed more servers than intended, and the 2021 Facebook outage came from a network configuration change.
2. **Nobody sees the whole system.** In a company, the person changing a security group or route table often doesn't know which other team's database or service depends on it. That's an **information-availability problem, not a skill problem**. Senior engineers know why they're changing something, but they can't see every other team's dependencies.
3. **Existing tools are either after the fact or too technical.** - AWS Config, CloudTrail, driftctl: report **after** the change is live. - `terraform plan`: shows the diff **before**, but as raw technical output (`~ update in-place`, `forces replacement`), with no dependency impact, no risk score and no rollback. - Checkov, tfsec: check each resource against policy rules **in isolation**. They don't trace how a change cascades to dependent resources.

> **Say:** "So the gap is a pre-deployment check that answers: what else does this change reach, how risky is it, how do I undo it, explained so anyone on the team can understand it, enforced automatically at the pull request."

---

3. What we built

A **GitHub Actions workflow** (`.github/workflows/blast-radius-check.yml`) plus a **Python analysis engine** (`analyzer/`) plus the **PR commenter** (`ci/pr_commenter.py`).

What the PR comment contains: 1. Header: ✅ **Passed (highest risk: X)** or ❌ **Blocked: risk reached CRITICAL** 2. Risk table: file, resource, action, impact, risk, number of affected resources, data completeness 3. **In plain English:** a 3-sentence summary anyone can read 4. **Dependency graph** (Mermaid flowchart, drawn by GitHub): changed resource in red, production or internet-facing resources in orange, others in blue, arrows showing where the change spreads, and **cloud-boundary arrows labelled like "Azure → GCP"** 5. **Recommendations** (collapsible) 6. **Rollback plan** (collapsible) 7. If blocked, the reason and the threshold

The comment is **updated in place** on every new push, not duplicated, the same way Infracost and Dependabot behave.

---

4. The method, named

Pre-deployment static blast-radius analysis of Infrastructure-as-Code.

- **Pre-deployment:** before `terraform apply`
- **Static:** reads the plan file; doesn't run or watch anything live
- **Blast-radius analysis:** traces how far a change spreads through dependent resources
- **Infrastructure-as-Code:** the input is a Terraform plan

The student side's counterpart is **Pre-execution dynamic blast-radius analysis of live API calls**: same concept, but the input is a live click or API call instead of a file.

---

5. The pipeline

```
CHANGE -> DEPENDENCIES -> IMPACT -> RISK -> EXPLANATION -> SAFE ACTION / ROLLBACK
```

Module	File	Job
1. Configuration change detection	`analyzer/plan_parser.py`	Reads the plan, finds each effective change and diffs before/after fields
2. Resource dependency analysis	`analyzer/graph_builder.py`	Builds a NetworkX directed graph of what depends on what
3. Impact assessment	`analyzer/impact_engine.py`	Blast radius: every reachable dependent, weighted by distance and criticality
4. Risk assessment	`analyzer/risk_engine.py`	Resource-type severity × action × sensitive fields × impact
5. Recommendations and history	`analyzer/recommendation_engine.py`, `analyzer/history.py`	Rule-based advice, stored analyses, similar past changes
Explanation	`analyzer/summary_generator.py`	Plain-English summary
Safe action	`analyzer/rollback_generator.py`	Undo steps computed before apply
Orchestration	`analyzer/pipeline.py`	Runs all of the above for each change, sorts riskiest first
Delivery	`ci/pr_commenter.py`	Builds the comment and Mermaid graph, posts it, sets pass/fail

---

6. Step by step: what happens when someone opens a PR

1. **Trigger.** The workflow runs `on: pull_request` when files matching `samples/**.json` or `**.tfplan.json` change.
2. **Permissions.** `contents: read`, `pull-requests: write`. Least privilege: it can read code and write a comment, nothing more.
3. **Job 1:** `test`**.** Installs dependencies and runs `pytest` (23 tests). If the analyser itself is broken, we don't trust its verdict.
4. **Job 2:** `analyze` (runs only if `test` passed, via `needs: test`). - Checks out with `fetch-depth: 0` (full history), so it can diff against the base branch. - `git diff --name-only origin/<base>...HEAD` finds the changed plan files. - Runs `python ci/pr_commenter.py <files>`: all files in one call, so there's **one aggregated comment**.
5. **Analysis** (for each plan file): `analyze_plan()` runs parser → graph → impact → risk → data completeness → recommendations → rollback → summary, saves to the database, and sorts results riskiest first.
6. **Comment.** It builds the Markdown body with a hidden marker `<!-- cloudguard-blast-radius-bot -->`. Using the GitHub REST API and the built-in `GITHUB_TOKEN`, it searches the PR's comments for that marker: if found, **PATCH** (update); if not, **POST** (create).
7. **Gate.** If the worst risk is in `CI_FAIL_ON_RISK` (default `"CRITICAL"`), the script exits with code 1, so the job fails and the PR gets a **red X**. With a **branch protection rule** that requires this check, the **merge button is blocked**. A team can set the threshold to `"HIGH,CRITICAL"`, or `""` to never block.

In a real company: the demo treats changed JSON files in `samples/` as "the plan". In production, you'd add steps that run `terraform plan -out tfplan` and `terraform show -json tfplan > change.tfplan.json`, and the bot picks up any `*.tfplan.json`.

---

7. How each module works (know the formulas)

Module 1 — Plan parser

- Input: output of `terraform show -json tfplan`, Terraform's **own documented, stable JSON format** (also used by Terraform Cloud and Atlantis). It's provider-agnostic, so AWS, Azure and GCP resources look the same. **That's what makes multi-cloud possible.**
- Reads `resource_changes[]`: address, type, actions, `before`, `after`, `after_unknown`.
- Maps actions: `["create"]` → create, `["update"]` → update, `["delete"]` → delete, `["delete","create"]` or `["create","delete"]` → **replace**. Skips `no-op` and `read`.
- `_diff_fields` compares before and after to list every changed field with old and new values.
- **Dependencies:** reads `configuration.root_module.resources[].expressions[].references` and `depends_on`. This is the **same reference data Terraform uses to decide apply order**, so the graph isn't guessed.
- **Walks nested modules recursively** (`module_calls`), and prefixes addresses the way Terraform does (`module.app.aws_instance.web`).

Module 2 — Graph builder

- **NetworkX DiGraph** (directed graph).
- If A references B (A uses B), the edge is stored as **B → A**, meaning "a change to B can spread to A".
- Each node gets metadata:
- `is_production`: tag `Environment` = `prod` or `production`
- `is_public` (internet-facing), checked **per cloud**: AWS `cidr_blocks` 0.0.0.0/0, Azure `source_address_prefix` `*` or `0.0.0.0/0`, GCP `source_ranges` 0.0.0.0/0, plus `publicly_accessible`, `map_public_ip_on_launch`, public ACLs, `public_network_access_enabled`.

Module 3 — Impact (blast radius)

- `nx.single_source_shortest_path_length(graph, changed)` is a breadth-first search that finds **every reachable dependent and its hop distance**, not just direct neighbours.
- **Weight of each affected resource:**

```
weight = (1  +  1.5 if production  +  1.0 if internet-facing) / (1 + hops)
impact_score = min(100, round(sum_of_weights × 12))
```

- **Impact level:** no dependents → LOW; sum < 2 → MEDIUM; sum < 5 → HIGH; otherwise → CRITICAL.
- **Why this design:** a change that reaches **one production database** should outrank one that reaches three untagged dev resources, and a resource 3 hops away is less likely to be affected than one directly attached.
- **The score is relative, not a probability.** 72 does **not** mean a 72% chance of failure. It ranks changes against each other.
- Also outputs the full blast-radius subgraph (every branch), which the Mermaid graph draws.

Module 4 — Risk

```
base severity  = HIGH   for networking/identity types (security_group, firewall, route_table, nat_gateway, iam, role_assignment,
kms, key_vault …)
               = MEDIUM for databases, storage, compute, VPC/subnet, load balancers, functions, clusters
               = LOW    otherwise
risk = max(base severity, impact level)
     + 1 level if a security-sensitive field changed (cidr_blocks, ingress, source_ranges, source_address_prefix, policy, acl,
publicly_accessible …)
     + 1 level if action is delete/replace and base severity isn't LOW
capped at CRITICAL
```

- Type keywords include AWS, Azure and GCP names side by side, so it works on a multi-cloud plan.

Worked example: `sg_open_ssh.json` Security group → base **HIGH**. Impact **HIGH** (3 dependents: EC2 instance, RDS database, load-balancer attachment). max = HIGH. The `ingress` field changed to 0.0.0.0/0 (sensitive), so +1 → **CRITICAL**. The gate fails and the PR is blocked.

Data completeness (called `confidence_score` in the code)

```
score = 55 + round(known_ratio × 42), capped at 97
known_ratio = 1 − (fields Terraform marks "unknown until apply") / (changed fields + unknown fields)
```

- It measures **how much of the change is knowable before apply**, for example a replaced database's new ID is unknown until apply, so `rds_delete.json` shows 87%.
- **Capped at 97, never 100**: the tool should never claim false certainty.
- **Our design decision:** an earlier version added points if similar changes had been seen before. We removed that, because **having seen something before doesn't make it safe now**: the environment, policies and dependencies may have completely changed. History is now only an informational lookup ("Similar past changes"), never part of the score. We also renamed it from "Confidence" to "**Data Completeness**" so it can't be misread as "safe".

Recommendations

- Resource-specific rules: SG opened to 0.0.0.0/0 → restrict CIDR or use a bastion or VPN. IAM policy change → least privilege, no wildcards. DB delete or replace → confirm backup and deletion protection. S3 ACL public → verify intent. Any delete → confirm no dependent still uses it.
- Plus one per risk level: CRITICAL → manual review, rollback plan, notify owners. HIGH → staging first and a maintenance window. MEDIUM → monitor for the first hour. LOW → normal review.

Rollback plan (computed **before** apply)

- create → `terraform destroy -target=<address>`
- delete → restore from the last known-good state or backup and re-apply (there's no partial undo)
- replace → revert config and re-apply, then update anything hardcoding the old ID
- update → "Set `<field>` back to `<old value>`" for each changed field
- Always ends with: run `terraform plan`, which should show no diff.

> **Say:** "Most tools stop at 'this is risky'. We give the undo steps before the change, so nobody has to improvise at 2 a.m. during an incident."

Plain-English summary

Three sentences: what the change does, how many resources it reaches (names up to 3), and how worried to be. Written for managers, clients and other teams.

Mermaid dependency graph

- GitHub renders ```` ```mermaid ```` blocks in comments as real diagrams. No image hosting, no extra service.
- Node labels: icon, type, name, cloud (AWS/Azure/GCP), CHANGED or "N hops away", production, internet-facing.
- **Cross-cloud edges are labelled** (for example `Azure → GCP`).
- Capped at **25 nodes** plus "...and N more" so the graph stays readable. The riskiest graph is expanded and the rest collapsed.

---

8. Real results on our sample plans (these are actual outputs)

Sample	Changed resource	Action	Impact	Risk	Affected	Data completeness	Gate
`sg_open_ssh.json`	`aws_security_group.web_sg`	update	HIGH	**CRITICAL**	3	97%	❌ Blocked
`rds_delete.json`	`aws_db_instance.app_db`	replace	HIGH	**CRITICAL**	2	87%	❌ Blocked
`iam_policy_widen.json`	`aws_iam_role_policy.lambda_policy`	update	MEDIUM	**CRITICAL**	1	97%	❌ Blocked
`multi_cloud_migration.json`	`azurerm_network_security_rule.dr_rdp_rule`	update	MEDIUM	**CRITICAL**	2 (Azure VM → GCP bucket)	97%	❌ Blocked
`route_table_change.json`	`aws_route_table.private_rt`	update	MEDIUM	**HIGH**	1	97%	✅ Passed (warns)
`log_retention_change.json`	`aws_cloudwatch_log_group.app_logs`	update	LOW	**LOW**	0	97%	✅ Passed

Tests: 23 automated tests pass (`python -m pytest tests/ -q`). They include one fixture that is **real, unedited** `terraform plan` **output**, not hand-written.

---

9. How we validated against real Terraform (a strong viva story)

> **Say:** "Our sample files were written by us, so an examiner could say: you only tested JSON shaped to fit your own code. So we generated a **real Terraform plan**: real Terraform, with the real HashiCorp AWS provider, pointed at **moto**, a fake AWS server that runs locally in Python. That cost nothing and needed no AWS account.
>
> That real plan **caught a real bug**. The config put the security group and EC2 instance inside a Terraform **module**, and our parser only read the root module, so it reported **0 affected resources** when the EC2 instance clearly used that security group. We fixed the parser to walk nested modules recursively, and added a test on that exact real file so the bug can't come back."

Why moto and not LocalStack? LocalStack runs in Docker. On Windows, Docker Desktop needs WSL2, and `wsl --install` failed for us with `Forbidden (403)` ; it also needed admin rights and restarts. moto's server mode is pure Python: `pip install "moto[server]"`, then `python -m moto.server -p 5001` . Same validation goal, far less setup, zero cost.

---

10. Tools and techniques, with why and why not

Tool	Why we used it	Rejected alternative, and why
Terraform plan JSON (`terraform show -json`)	Terraform is the most widely used multi-cloud IaC tool. The plan JSON is documented and stable, has before/after values and the reference graph, and is **provider-agnostic, so multi-cloud comes for free**	**Parsing raw** `.tf` **files:** no resolved values or actions. **CloudFormation:** AWS-only.
Python 3.12	Readable, strong libraries for graphs and testing, same language as the student CLI guard	
NetworkX	Mature graph library: directed graphs, BFS hop distances, shortest paths, subgraphs	**Hand-written BFS:** reinventing a tested wheel. **Neo4j graph database:** a server to run, overkill for one plan.
Rule-based risk engine	Every verdict traces to a statable rule: explainable, deterministic, auditable	**ML or AI:** no labelled dataset of "this change caused this outage", and a black box is hard for engineers to trust
GitHub Actions	Runs where the decision happens (the PR), on the company's own runner, and can block merges through branch protection	**A dashboard:** people forget to check it, and it can't block anything. We built one first and dropped it for this reason.
Built-in `GITHUB_TOKEN`	Auto-created per run, scoped to the repo, expires after the job, no secrets to manage	A personal access token: long-lived, broader access
GitHub REST API via `urllib` (standard library)	No extra dependency means a smaller supply-chain surface	`requests` or PyGithub add packages for two HTTP calls
Hidden comment marker + PATCH	One comment updated in place, no spam	A new comment each push
Mermaid	GitHub renders it natively in comments: a real diagram with no image hosting	**Graphviz or PNG images:** need hosting or upload. **The Flask dashboard:** dropped.
SQLAlchemy + SQLite (MySQL optional via `schema.sql`)	Stores each analysis and powers "similar past changes". SQLite needs zero setup; MySQL was in the original report. SQLAlchemy lets us switch with one URL.	
pytest	23 tests; the workflow runs them **before** analysing, so a broken analyser never gives a verdict	
moto[server]	Generates a genuine Terraform plan locally for free (see section 9)	LocalStack (Docker/WSL2 problems)
boto3 + read-only IAM policy (optional)	Optional live enrichment (tags) with exactly 3 `Describe*` permissions. Even a bug can't change anything.	Broad credentials

What changed from our original proposal, and why

The original proposal used **CloudTrail + AWS Config + EventBridge + Lambda + SNS** to monitor a live AWS account, with a dashboard. - **Cost:** AWS Config bills per configuration item and per rule evaluation from the first item, and a CloudTrail trail needs an S3 bucket. We had a **zero-billing requirement** (my friend's surprise AWS bill is why). - **Timing:** that pipeline reacts **after** a change happens, which is exactly the limitation of existing tools. - **Dashboard → PR bot:** engineers don't open dashboards before every change. A check on the PR is automatic and can actually block.

So we moved from **after-the-fact monitoring** to **before-the-fact analysis**, which is both free and more useful.

---

11. Comparison with existing tools

> **Say:** "Parts of this idea exist, and we studied them. What's different is the combination, and the fact that no one's credentials leave their own pipeline."

Tool	What it does	How CloudGuard differs
`terraform plan`	Shows the diff before apply	Raw technical text. No dependency impact, risk score, rollback or plain English. We **build on** it: it's our input.
AWS Config, CloudTrail, driftctl	Record and detect changes or drift	**After** the fact. AWS Config costs money.
Checkov, tfsec	Static policy checks on IaC, all three clouds	Check each resource **in isolation**. They don't trace how a change **cascades** through dependents or across clouds.
Infracost	PR comment with **cost** impact	Cost, not risk or dependencies. (We copied its good UX: one comment updated in place.)
Atlantis, Terraform Cloud	Run plan and apply from PRs	Run the plan, don't analyse its blast radius
Open-source "Blast Radius" (28mm)	Interactive visualisation of the Terraform dependency graph, run locally	A visualiser: no risk scoring, recommendations, rollback, or CI gate
Overmind	Commercial SaaS that calculates blast radius and risks of Terraform changes, using discovery of your **live** infrastructure	Needs **standing read access to your cloud account**, held by a third party. CloudGuard runs **inside your own CI on the plan file**: no third party ever holds credentials. Free and open. Adds rollback steps, a plain-English summary and a student side. **Honest point:** Overmind sees live resources outside the plan, which we don't (yet).

Your credential-security argument, phrased correctly:

> **Say:** "Unlike SaaS tools such as Overmind, which need standing access to customers' cloud accounts, CloudGuard analyses the Terraform plan inside the user's own CI pipeline. No third party holds credentials, so there's no central vendor whose breach could expose many customers at once. The remaining risks are keeping plan files private, because plans can contain secrets, and pinning CI dependencies. We'd address both in production."

(Don't group the open-source Blast Radius with Overmind on credentials; it runs locally too. And say "access to accounts", not "account IDs".)

---

12. Live demo plan

Before the viva (checklist): - Repo `Shobaa-cloud/HybridCloudImpactAnalyzer_Bot`. The bot lives on branch `demo/blast-radius-check`. (`main` still holds the old prototype, so either demo from that branch or merge it into `main` first.) - Keep the existing PR from `test-critical-sg` into `demo/blast-radius-check`, which shows **CRITICAL + red X**. - Do the **safe-change test** too: a PR that only edits `samples/log_retention_change.json` should show **LOW + green check**. Showing both proves the gate isn't just always red.

Demo: 1. Open the CRITICAL PR. Show the ❌ **Blocked** header and the risk table. 2. Read the **plain-English** line out loud. 3. Show the **dependency graph**: security group in red → EC2, RDS and LB attachment. 4. Expand **Recommendations** and **Rollback plan**. 5. Scroll to checks: the **red X on** `analyze`. Explain that branch protection turns this into a blocked merge. 6. Open the **Actions** tab to show the `test` job then the `analyze` job logs. 7. Open the safe PR: ✅ **Passed**, LOW, "No other resources depend on...". 8. Optional: show `samples/multi_cloud_migration.json`'s graph with the **Azure → GCP** labelled arrow.

Offline backup (if the internet fails):

```
python ci/pr_commenter.py samples/sg_open_ssh.json          # prints the comment, exit code 1
python ci/pr_commenter.py samples/log_retention_change.json # exit code 0
python -m pytest tests/ -q                                  # 23 passed
```

---

13. Advantages

1. **Pre-deployment:** catches the problem at the PR, before `apply`, not in production (**shift-left**).
2. **Real dependency graph:** built from Terraform's own reference data, including nested modules, not a guess or a count.
3. **Weighted blast radius:** every reachable dependent, discounted by distance, boosted for production and internet-facing resources.
4. **Truly multi-cloud:** AWS, Azure and GCP rules side by side, and it traces changes **across** clouds in one graph.
5. **No credentials, no third party:** runs on the plan file inside the user's own CI.
6. **Zero cost:** no AWS Config, no live monitoring, no servers. GitHub Actions minutes are free for public repos.
7. **Enforced, not advisory:** fails the check and blocks the merge on CRITICAL, with an adjustable threshold.
8. **Rollback ready before apply.**
9. **Plain English for non-engineers:** managers and clients can follow it.
10. **Visual:** dependency graph drawn in the PR with cross-cloud labels.
11. **Explainable and deterministic:** same input, same verdict, and every verdict traces to a rule.
12. **Honest metrics:** "Data completeness" instead of an inflated "confidence", and the impact score is explicitly relative, not a probability.
13. **Tested against real Terraform output**, which caught and fixed a real module bug.
14. **Low friction:** one workflow file, one comment updated in place, no spam.

---

14. Limitations (say these confidently)

1. **Only sees what's in the plan.** If a resource outside this Terraform project (say, created by hand in the console) depends on the changed one, it won't appear. Fix: merge a read-only live-state snapshot into the graph (partly built in `live_state.py`, not wired in).
2. **The demo trigger is sample files.** In a real company, a `terraform plan` + `terraform show -json` step is needed in the workflow.
3. **Rule-based severity is a heuristic.** The keyword lists and weights (1.5, 1.0, ×12, thresholds 2 and 5) are project-defined, not learned from incident data. They're tunable.
4. **The impact score is relative**, not a failure probability.
5. **History doesn't persist in CI.** Each Actions run starts with a fresh SQLite database, so "similar past changes" needs a persistent database (via a secret) to be useful there.
6. **Mermaid limits:** a static picture (no zoom or click), capped at 25 nodes, and a PR comment is limited to about 65,000 characters.
7. **Plan files can contain secrets** in plain text, so real plans must not be committed. Better to generate them inside the job as artifacts.
8. **CI supply-chain risk:** third-party actions and pip packages. Pin versions or commit SHAs in production.
9. **Terraform only:** no CloudFormation, Pulumi or ARM/Bicep.
10. **Module outputs** like `module.network.vpc_id` are resolved best-effort to the module, not always to one exact resource.
11. **Optional live AWS features** (`aws_enrichment.py`, `live_state.py`) are scoped read-only but weren't run against a real account in this build.

---

15. Question bank (practise these out loud)

Q1. Explain your project in one line. A PR bot that reads a Terraform plan before it's applied and tells the team what else the change will break, how risky it is, and how to undo it, and blocks the merge if it's critical.

Q2. What is a Terraform plan? The preview Terraform generates of what `apply` would create, update, replace or delete. `terraform show -json` gives it as structured JSON, which is our input.

Q3. What is blast radius? The set of resources a change can affect beyond the one being changed. Change a security group, and every instance and database behind it is in the blast radius.

Q4. `terraform plan` already shows changes before apply. Why is your tool needed? Plan shows **what** changes, not **what else is affected** or **how risky** it is. It's a technical diff. We add dependency tracing, weighted impact, risk, plain English, a graph, recommendations, rollback and an enforced gate. Plan is our input, not our competitor.

Q5. Professional engineers know what they're changing. Why would companies need this? They know **their** change, but not every other team's dependency on a shared resource. That's information availability, not skill. Also, reviewers can't manually trace every hop in large plans

every day. Automation doesn't replace their judgment; it gives them the missing information at review time.

Q6. How do you find dependencies? From the plan's `configuration` section: each resource's `expressions[].references` and `depends_on`. That's the same data Terraform uses to order the apply. We walk nested modules recursively and build a NetworkX directed graph.

Q7. Why is the edge direction B → A when A references B? Because the question we ask is "if B changes, who is affected?". A uses B, so a change to B flows to A. BFS from the changed node then finds everything downstream.

Q8. Explain your impact score. Each reachable dependent contributes (1 + 1.5 if production + 1 if internet-facing) ÷ (1 + hops). Sum them, multiply by 12, cap at 100. Levels: no dependents LOW, under 2 MEDIUM, under 5 HIGH, otherwise CRITICAL. It's relative, for ranking, not a probability.

Q9. Why divide by (1 + hops)? The further away a resource is, the less directly it's affected. One hop gives half weight, two hops a third.

Q10. Explain your risk calculation. Base severity from resource type (network/identity HIGH; DB/compute/storage MEDIUM; else LOW). Take the higher of that and impact level. Bump one level if a security-sensitive field changed, and one more for delete or replace on a non-LOW type. Cap at CRITICAL.

Q11. Why is `sg_open_ssh` CRITICAL? Security group base HIGH, impact HIGH with 3 dependents, and the `ingress` field changed to 0.0.0.0/0, which is sensitive, so bumped to CRITICAL.

Q12. Why is the IAM change CRITICAL with only 1 affected resource? IAM is base HIGH on its own because permissions are high-stakes, and the policy field (sensitive) changed to `Action: "*"`, which bumps it to CRITICAL. Risk isn't only about how many resources are reached.

Q13. What's "data completeness"? Why not call it confidence? The share of the change that's knowable before apply, versus what Terraform marks "unknown until apply". 55 to 97, never 100. It used to add points for having seen similar changes before. I questioned that: the environment may have completely changed since then, so familiarity doesn't mean safety. We removed it and renamed the score so nobody reads it as "safe".

Q14. Then why keep history at all? As an informational lookup for humans, like a doctor checking family history: it's context, not the diagnosis. It shows "similar past changes and their outcomes" but never affects the score.

Q15. How does it block the merge? The script exits with code 1 when the worst risk is in `CI_FAIL_ON_RISK`. The job fails, the PR gets a red X, and a branch protection rule requiring the `analyze` check disables merging. An authorised approver can override if it's intentional.

Q16. How do you avoid spamming comments? A hidden HTML marker in the comment. On each run we search the PR's comments for it and PATCH that comment instead of posting a new one.

Q17. How does the bot authenticate to GitHub? Is it secure? With the built-in `GITHUB_TOKEN`, created per run, scoped to the repo, expiring when the job ends. The workflow grants only `contents: read` and `pull-requests: write`.

Q18. Does it need AWS, Azure or GCP credentials? Not for the core analysis. It reads the plan file only. In a real pipeline, generating the plan needs credentials, but those stay in the company's own CI. The optional live AWS enrichment uses a read-only policy with exactly three `Describe*` actions.

Q19. How is it better than Overmind? Overmind is a hosted service that needs standing access to your cloud accounts. We run inside your own pipeline with no third-party access, it's free and open, and we add a rollback plan, plain-English summary and a student side. Overmind's strength is that it sees live infrastructure beyond the plan; that's our main future-work item.

Q20. How is it different from Checkov or tfsec? They check each resource against security policies in isolation ("this SG allows 0.0.0.0/0"). We trace consequences: this SG change reaches this EC2 instance and this production database, across clouds, with a rollback. They're complementary: a team could run both.

Q21. Why is this "hybrid" or multi-cloud? Terraform's plan format is the same for all providers, and our rules include AWS, Azure and GCP equivalents (for example security_group, network_security_rule, firewall; cidr_blocks, source_address_prefix, source_ranges). `multi_cloud_migration.json` shows an Azure firewall rule change cascading to an Azure VM and then a GCP bucket, and the graph labels the `Azure → GCP` edge.

Q22. Why NetworkX? A mature, tested graph library. `single_source_shortest_path_length` gives every reachable node with its hop count in one BFS, and subgraphs give us the drawing. A graph database would be a server to run for a graph that fits in memory.

Q23. Why rule-based and not machine learning? No labelled dataset of real change-to-outage cases exists to train on. Rules are explainable, deterministic and auditable, which matters for a tool that blocks merges. ML is future work if incident data were collected.

Q24. Why a PR bot and not a dashboard? We first built a Flask dashboard, then dropped it. Dashboards depend on someone remembering to check. The PR is where the decision is made, and a CI check can actually enforce it.

Q25. Why Mermaid for the graph? GitHub renders Mermaid natively in comments, so we get a real diagram with no image hosting or external service. The limit is that it's static, so we cap it at 25 nodes.

Q26. Why did you move away from the CloudTrail + Config + Lambda design in your proposal? Cost (AWS Config bills per item and evaluation, and a CloudTrail trail needs S3) and timing (it only reacts after a change). The pre-deployment design is free and catches problems earlier.

Q27. How did you test it? 23 pytest tests: parsing, diffing, graph edges, transitive blast radius, risk bumps, multi-cloud crossing, rollback, summaries, module nesting, graph branches, and the Mermaid cross-cloud label. One fixture is real Terraform output generated against moto. The workflow runs the tests before every analysis. End to end, a PR on GitHub gave CRITICAL and a red X as expected.

Q28. What's moto, and why not LocalStack? moto fakes AWS APIs locally in Python. We pointed real Terraform at it to generate a genuine plan at zero cost. LocalStack needs Docker, which on Windows needs WSL2, and `wsl --install` failed with 403 on our machine. moto needed just pip.

Q29. What's the biggest bug you found? The module bug: a real Terraform plan with resources inside a module reported 0 affected resources, because we only read the root module. Found by testing real output, fixed by recursive module walking, locked in by a test.

Q30. What's your accuracy percentage? It's deterministic rules, not a statistical model, so there's no accuracy percentage in the ML sense. On all 6 samples and the real Terraform fixture, it produced the expected dependencies and risk levels, and all 23 tests pass. Where it could be wrong: dependencies outside the plan, and severity weights that are heuristics. Those are the limitations we'd address next.

Q31. Where does your project lag? It can't see resources outside the plan, the demo trigger isn't a real `terraform plan` step, history doesn't persist in CI, and the weights aren't calibrated on real incident data.

Q32. Who benefits? Cloud, DevOps and SRE teams using Terraform, especially where several teams share infrastructure. Also reviewers and managers, thanks to the plain-English summary, and interns and freshers joining such teams.

Q33. How would a company adopt it? Copy `analyzer/`, `ci/`, `db/`, `config.py`, `requirements.txt` and the workflow into their Terraform repo. Add a `terraform plan` + `terraform show -json` step. Require the `analyze` check in branch protection. Tune `CI_FAIL_ON_RISK`.

Q34. Can plan files leak secrets? Yes. Plans can include sensitive values in plain text. In production, generate the plan inside the job and never commit it. Our committed samples are fake data for the demo.

Q35. How does this connect to the student side? Same idea (blast radius before the change) with a different input. Professionals: a Terraform plan in a PR (static). Students: the live console click or API call (dynamic). The website asks the user's role and sends them to the right one.

Q36. Is it AI? No. It's deterministic, rule-based automation, by design, so every verdict is explainable.

Q37. Does it watch the account 24/7? No. It runs when a PR changes a plan file. That's event-driven, and it's why it costs nothing.

Q38. Did you use AI tools while building it? Answer honestly: yes, as a coding assistant, like most developers today. But the problem, the design decisions and the testing were ours, and I can walk through every module. Then give examples from the note at the end of this file.

Q39. What would you add with more time? A real `terraform plan` step in CI; merge a read-only live snapshot into the graph so dependencies outside the plan appear; a persistent database for history; calibrate weights on real incident data; support CloudFormation and Pulumi; publish it as a reusable GitHub Action.

---

16. Phrases to use, and traps to avoid

Don't say	Say instead
"Terraform plan shows changes only after"	"Terraform plan shows a technical diff, but not what else breaks or how risky it is"
"Impact score 72 means 72% chance of failure"	"It's a relative score for ranking changes"
"Confidence score"	"Data completeness"
"It's live testing" / "monitors 24/7"	"It's static analysis of the plan before deployment, triggered by the PR"
"It's AI"	"It's rule-based, which is deliberate for explainability"
"It's 100% secure"	"No third-party credential access; the remaining risks are plan secrets and CI dependencies"
"Nothing like this exists"	"Parts exist; the combination, and doing it without third-party access, is what's different"
"Other tools leak account IDs"	"Hosted tools hold access to customers' accounts"

---

17. Key numbers to remember

- **23** tests pass. **6** sample plans plus **1** real Terraform fixture.
- Weights: base **1**, production **+1.5**, public **+1.0**, ÷ **(1 + hops)**, × **12**, cap **100**. Thresholds **2** and **5**.
- Data completeness: **55** to **97**, never 100.
- Gate default: **CRITICAL**. Graph cap: **25** nodes.
- Workflow jobs: **test** → **analyze**. Permissions: **contents: read**, **pull-requests: write**.

---

18. Code map

File	What it does	Key names
`.github/workflows/blast-radius-check.yml`	Trigger, permissions, `test` then `analyze`, changed-file detection	`CI_FAIL_ON_RISK`, `needs: test`, `fetch-depth: 0`
`ci/pr_commenter.py`	Runs the pipeline, builds the comment and Mermaid graph, POST or PATCH, exit code gate	`build_comment_body`, `build_mermaid_graph`, `COMMENT_MARKER`, `post_or_update_comment`, `MAX_GRAPH_NODES`
`analyzer/pipeline.py`	Orchestrates every module per change, sorts riskiest first	`analyze_plan`
`analyzer/plan_parser.py`	Module 1: changes, diffs, references, nested modules	`parse_resource_changes`, `parse_dependency_references`, `_walk_module`, `ACTION_MAP`
`analyzer/graph_builder.py`	Module 2: NetworkX DiGraph with production and public metadata per cloud	`build_dependency_graph`, `_extract_node_metadata`
`analyzer/impact_engine.py`	Module 3: weighted blast radius, chain, subgraph	`assess_impact`, `_node_weight`
`analyzer/risk_engine.py`	Module 4: severity × sensitive fields × action × impact	`assess_risk`, `HIGH_SEVERITY_TYPES`, `SENSITIVE_FIELD_KEYWORDS`
`analyzer/recommendation_engine.py`	Rule-based recommendations	`generate_recommendations`
`analyzer/rollback_generator.py`	Undo steps before apply	`generate_rollback_plan`
`analyzer/summary_generator.py`	Plain-English summary	`generate_plain_summary`
`analyzer/history.py`	Saves analyses, similar past changes, data completeness	`compute_confidence_score`, `find_similar_past_changes`
`db/models.py`, `schema.sql`	SQLAlchemy models; MySQL schema option	
`config.py`	Database URL, score bounds (55, 97)	
`analyzer/aws_enrichment.py`, `analyzer/live_state.py`, `iam-readonly-policy.json`	Optional read-only live AWS enrichment	
`tests/test_pipeline_components.py`, `tests/fixtures/real_terraform_module_plan.json`	23 tests; real Terraform fixture	`test_module_nested_dependency_is_traced`
`samples/*.json`	6 demo plans across the risk spectrum	
`app.py`, `templates/`, `static/`, `cli.py`	Earlier dashboard and CLI (not used in the final demo)	

Note for you (not for the examiner)

These decisions came from **your own questions**. Use them to show you own the project: - "Why add confidence from history? The environment may have changed." That's why the score became **data completeness**. - "Pros already know what they change." That led to the **information-availability** answer (Q5). - "I don't want a dashboard, I want the bot." That led to the **PR bot as a gate**. - "Where's the

dependency graph?" That led to the **Mermaid graph in the PR**. - "No one's credentials, so no vendor breach risk." That led to the **zero third-party access** advantage. - "I won't be billed a single penny." That led to **no AWS Config, and moto instead of a live account**. - "Why not moto instead of LocalStack?" led to the real-Terraform validation that **caught the module bug**.